

Rapport de Projet IA S6

Maelenn Piat

Manech Bourgeois

Gaspard Vieujean

08/06/2026 - 12/06/2026

Lien : https://github.com/Swe3fty/projet_ia_s6.git

Contexte du projet

Dans le cadre de notre formation FISE3 à l'ISEN Brest, nous avons réalisé ce projet d'IA qui a pour but de consolider et de mettre en pratique nos connaissances en Intelligence Artificielle. Pour ce faire, nous exploitons un jeu de données généré lors de notre précédent projet de Big Data, regroupant les informations issues de la base nationale des Infrastructures de Recharge pour Véhicules Électriques (IRVE). Afin de valoriser ces données, il nous est imposé d'appliquer différentes méthodes de Machine Learning, allant de la préparation et la visualisation des données jusqu'à l'apprentissage non supervisé et supervisé.

Pour satisfaire ces objectifs et répondre aux besoins du client, il nous est demandé de réaliser une visualisation sur carte intégrant un regroupement automatique des points de recharge par type d'implantation, ainsi qu'une carte de chaleur. Il est également attendu un clustering basé sur la position géographique des bornes. Enfin, le volet prédictif requiert le développement de deux modèles d'IA : l'un pour prédire le type d'implantation, et l'autre pour estimer la puissance nominale.

L'ensemble de ces besoins se traduit par 5 fonctionnalités principales, que nous nous sommes répartis afin d'optimiser notre efficacité. Pour cela, l'élaboration d'un diagramme de Gantt (que vous trouverez en page suivante) nous a été très utile pour nous organiser. Il est à noter que notre méthode de travail est restée agile : nous avons adapté notre organisation au fur et à mesure et ajusté les dépendances en fonction de l'état d'avancement du projet.

Diagramme de Gantt :



Pour mener à bien ce projet sur une période extrêmement condensée de cinq jours (du 8 au 12 juin), l'élaboration et le suivi d'un diagramme de Gantt ont été indispensables.

Dès le premier jour, nous avons opté pour une stratégie de parallélisation des tâches techniques. La charge de travail a été divisée pour permettre à chacun d'avancer simultanément sur une fonctionnalité clé : Gaspard a pris en charge le développement de la visualisation sur carte, Manech s'est concentré sur les algorithmes de clustering, tandis que Maelenn a initié la conception du modèle de prédiction lié au type d'implantation.

Notre planification met également en évidence une gestion des dépendances techniques. Par exemple, le développement du second modèle (prédiction de la puissance nominale) a été enchaîné par Manech à la suite du clustering. De la même manière, la préparation du support de soutenance n'a débuté qu'en fin de semaine, une fois l'essentiel du code validé et les résultats de nos prédictions consolidés.

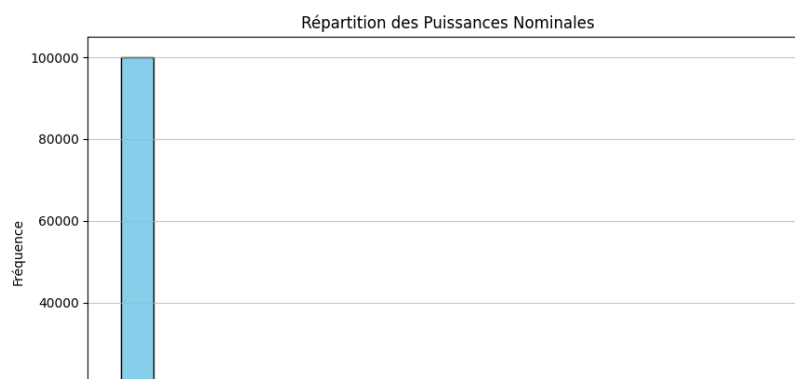
Enfin, pour garantir un rendu de qualité, nous avons fait le choix de ne pas repousser la documentation à la veille de la livraison. Cette méthode agile nous a permis de nous adapter au fur et à mesure.

Besoin client 1 : Visualisation sur carte

Nettoyage des données et organisation

Afin d'identifier et de localiser efficacement les points de charge, nous avons fait le choix de réduire le jeu de données de 53 à 11 colonnes essentielles. Nous avons conservé les coordonnées géographiques (lat, long), la puissance_nominale, l'implantation et les caractéristiques des prises, ce qui a notamment permis d'optimiser le chargement en réduisant l'utilisation de la mémoire de 54.4 MB à 7.1 MB.

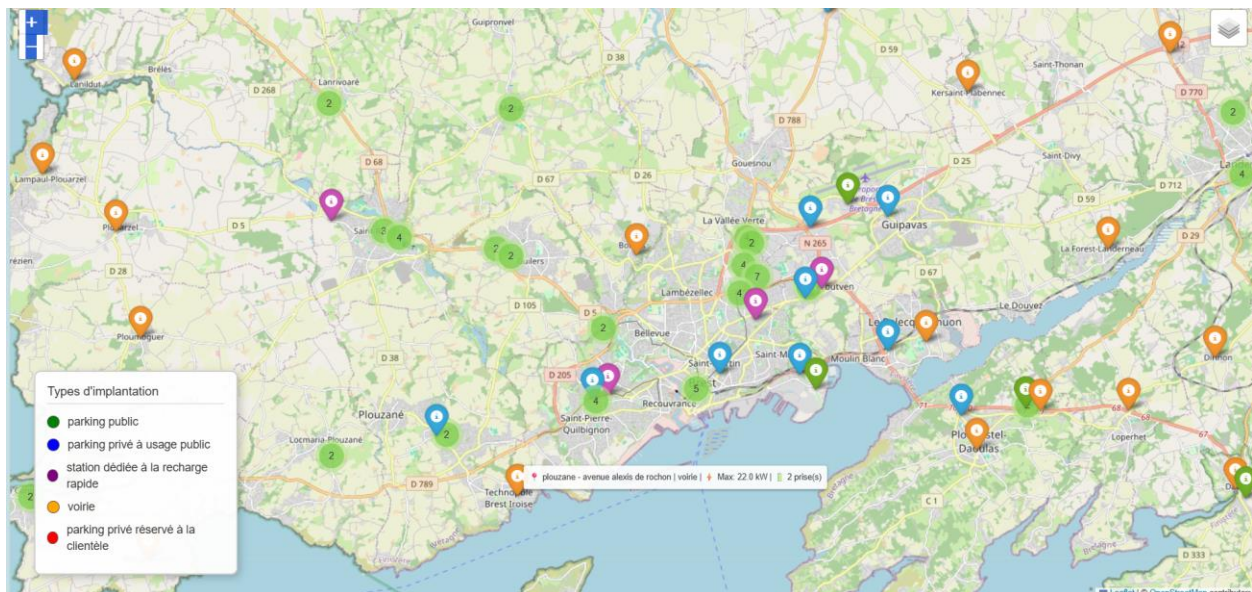
Pour obtenir des données normalisées cela a nécessité plusieurs étapes de nettoyage. Nous avons d'abord supprimé les lignes présentant des valeurs manquantes pour la latitude, la longitude ou l'implantation à l'aide de la fonction dropna. Ensuite, un regroupement géographique (groupby) a été appliqué pour fusionner les différentes bornes appartenant à une même station, évitant ainsi la superposition inutile de points. Enfin, un nettoyage des chaînes de caractères (retrait des apostrophes, guillemets et retours à la ligne) a été réalisé via des expressions régulières pour assurer la bonne exécution du code Javascript lors de l'affichage. Avec tout cela nous avons pu finalement visualiser les données avec des graphiques en commençant par la répartition des Puissances.



On remarque majoritairement que les puissances se retrouvent entre 0 et 50kW, ceci s'explique par la normalisation des bornes.

Génération des cartes

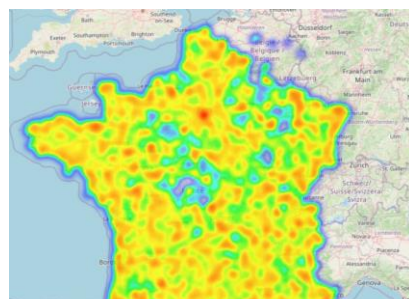
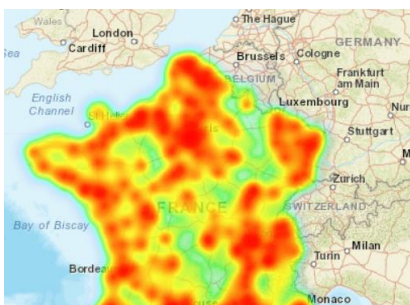
Pour la génération des cartes interactives, notre choix s'est porté sur la bibliothèque Folium. Afin de maintenir une carte lisible et fluide, la classe MarkerCluster a été implémentée pour regrouper dynamiquement les stations selon leur type d'implantation lors du dézoom.



Carte de visualisation des types d'implantation

Pour la carte dédiée aux puissances, les bornes ont été discrétisées en trois catégories métiers (Normale, Rapide et Extrêmement Rapide) via la fonction `pd.cut` (non représentée ici car elle reprend les mêmes points que la carte d'avant).

Pour clôturer ce besoin, l'extension HeatMap a été utilisée pour illustrer les zones de forte concentration géographique et représenter visuellement la densité spatiale des infrastructures.



HeatMap des bornes de recharge (Projet Big Data)

HeatMap des bornes de recharge (Projet IA)

En comparant les deux HeatMap celle du Projet IA et Projet Big data on remarque que les grandes zones de chaleurs sont dans les villes et majoritairement aux extrémités de la France avec des exceptions. Il y a beaucoup de ressemblance au niveau des données.

Besoin client 2 : Clustering selon la position

Préparation des données et choix des variables

Dans le cadre du besoin client 2, l'objectif est de regrouper les bornes de recharge selon leur position géographique. Pour répondre à cet objectif les deux variables utilisées sont latitude et longitude. L'ajout d'autres données n'apporte rien à l'analyse et aurait faussé le résultat du clustering.

Comme pour le premier besoin, les données ont été nettoyées en faisant attention aux territoires d'outre-mer (Guadeloupe, La Réunion, etc) vu que leurs coordonnées géographiques sont très éloignées du territoire. De plus, un regroupement des données par station a été fait.

Choix de l'algorithme et principe de fonctionnement

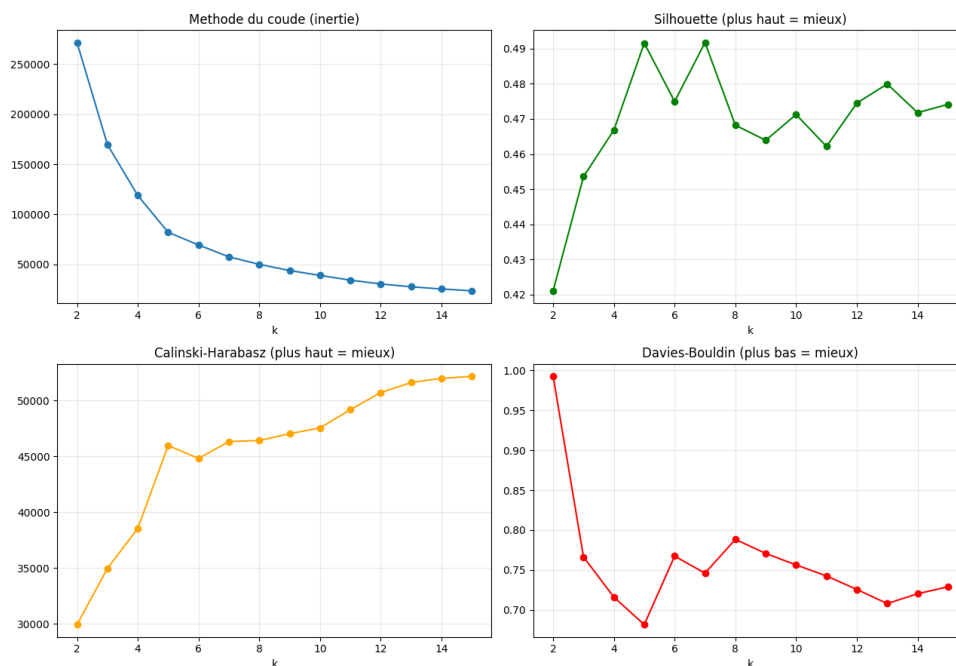
L'algorithme choisit ici est K-Means. Son principe repose sur la minimisation de l'inertie intra-cluster, c'est-à-dire la somme des distances au carré entre chaque point et le centre de son cluster. L'algorithme fonctionne de manière itérative en deux étapes : une affectation et une étape de mise à jour. Les deux calculent la distance euclidienne avec le centroïde. Ces étapes sont répétées jusqu'à ce que le résultat soit le même.

Le choix de l'algorithme est justifié par plusieurs raisons, il est particulièrement adapté aux données géographiques continues, où la notion de distance euclidienne entre coordonnées est directement interprétable à l'échelle nationale. Ensuite, il est efficace et scalable sur des datasets de grande taille. Et enfin, il produit des clusters de forme convexe cohérents avec un découpage en région de la France.

Recherche du nombre optimal de clusters

K-Means nécessite de spécifier le nombre de clusters k avant l'entraînement. Pour déterminer la valeur optimale, plusieurs valeurs de k ont été testées de 2 à 15, et évaluées à l'aide de quatre métriques complémentaires.

La méthode du coude : tracer l'inertie en fonction de k et identifier le point d'inflexion à partir duquel l'ajout d'un cluster supplémentaire n'apporte plus de gain. Le coefficient de silhouette mesure pour chaque point s'il est bien assigné à son propre cluster par rapport à ses voisins (1 bonne affectation et -1 l'inverse). L'indice de Calinski-Harabasz mesure le rapport entre la dispersion inter-cluster et la dispersion intra-clusters. Et pour finir, l'indice de Davies-Boudin calcule la similarité moyenne entre chaque cluster et son voisin le plus proche. Les résultats obtenus sont présentés dans le tableau et les graphes ci-dessous :



Graphes des indices pour obtenir le nombre de clusters

En analysant les résultats, on retient $k = 5$ comme nombre optimal de clusters d'après les différents indices. En effet la majorité ont un changement d'allure de la courbe à partir de 5.

Visualisation et script de prédiction

Les résultats sont visualisés sur une carte interactive générée avec Folium. Les différents clusters sont représentés avec les différentes couleurs.

On remarque bien les 5 couleurs et donc les 5 clusters.

Notre modèle marche. Conformément aux exigences du cahier des charges, un script de prédiction a été



développé. Le modèle est donc entraîné une seule fois et sauvegardé au format .pkl via la bibliothèque joblib.

Carte Folium avec les différents clusters

Besoin Client 3 : Modèle de prédiction du type d'implantation

Préparation des données et choix des variables

L'objectif du besoin client 3 est de développer un modèle d'intelligence artificielle capable de prédire le type d'implantation d'une borne de recharge électrique à partir de ses caractéristiques. La variable cible est *implantation_station*, qui représente le type de lieu où la borne est installée (parking, voirie, etc).

Les variables explicatives ont été sélectionnées en tenant compte de leur lien logique avec le type d'implantation. Les variables retenues sont les suivantes : le nombre de points de charge (*nbre_pdc*), la puissance nominale (*puissance_nominale*), la localisation géographique (*lat*, *long*), les types de prises disponibles (*prise_type_ef*, *prise_type_2*, *prise_type_combo_ccs*, *prise_type_chademo*, *prise_type_autre*), les modalités de paiement (*paiement_acte*, *paiement_cb*), la possibilité de réservation (*reservation*), l'accessibilité aux deux-roues (*station_deux_roues*), la condition d'accès (*condition_acces*) et l'accessibilité PMR (*accessibilite_pmr*).

Ces variables ont été retenues car elles caractérisent directement le contexte fonctionnel et géographique d'une borne, qui est très lié à son lieu d'implantation. Par exemple, une borne à haute puissance avec prise combo CCS et paiement par carte bancaire aura plus de probabilité d'être implantée sur une voirie ou dans une station dédiée qu'une borne domestique de faible puissance. A ne pas oublier que la longitude et latitude indiquent la zone d'implantation.

Concernant le prétraitement, les variables booléennes ont été normalisées. D'autres catégories ont été converties en minuscules et nettoyées. L'ensemble du prétraitement a été sauvegardé dans un fichier *preprocessor.pkl*, comme demandé dans le cahier des charges, afin d'être réutilisé à l'identique lors des prédictions sans relancer la phase d'apprentissage. De plus, le *label_encoder_target.pkl* de la cible (*implantation_station*) a également été sauvegardé séparément en libellés lisibles (0,1, etc). Les noms de colonnes utilisés à l'entraînement ont été conservés dans *feature_names.pkl* pour garantir la cohérence de l'ordre des variables en entrée du modèle.

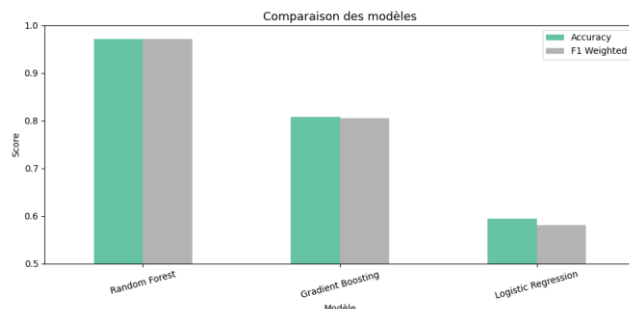
Choix de l'algorithme et principe de fonctionnement

L'algorithme retenu est Random Forest (Forêt aléatoire), dont le modèle entraîné est sauvegardé sous le nom *best_model_rf.pkl*. Il s'agit d'un algorithme d'apprentissage supervisé de type d'ensemble, qui construit un grand nombre d'arbres de décision indépendants lors de

la phase d'entraînement, puis simplifie leurs prédictions par vote majoritaire pour la classification.

Chaque arbre est entraîné sur un sous-échantillon aléatoire du jeu de données (bootstrap) et utilise à chaque nœud un sous-ensemble aléatoire des variables disponibles pour trouver la meilleure division. Cette méthode rend le modèle robuste au surapprentissage, ce qui est un avantage par rapport à un arbre de décision simple. Finalement le modèle fournit les probabilités d'appartenance à chaque classe via *predict_proba*.

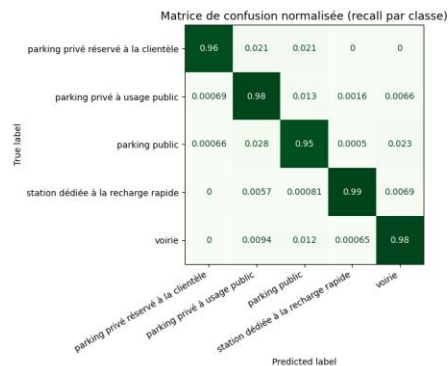
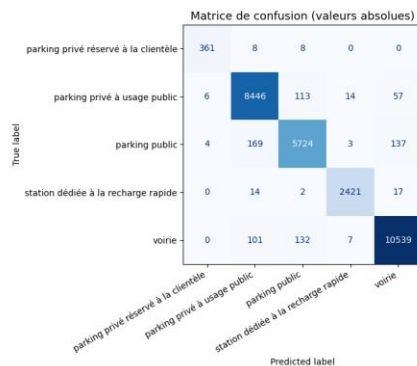
Nous avons fait le choix d'utiliser ce modèle pour plusieurs raisons. Il excelle sur des données mixtes (numérique et catégorielles), il résiste aux valeurs aberrantes et fournit une feature d'importance utile pour la justification. Avant de le choisir, nous avons pu comparer avec des modèles de Gradient Boosting et de Régression logistique, grâce au *f1_score* (métrique d'évaluation de la performance d'un modèle de classification) nous avons pu sortir un graphique de comparaison représenté ci-dessous :



Histogramme de comparaison des modèles avec le *f1 score*

On remarque que le plus précis d'entre eux est évidemment celui qu'on a choisi : le Random Forest. A noter que le *f1 score* est la moyenne harmonique de la précision et du rappel, particulièrement utilisé en cas de déséquilibre entre les classes.

De plus, une matrice de confusion a également été produite pour visualiser les erreurs de classification entre les différents types d'implantation. Le résultat obtenu est représenté juste en dessous.



Matrice de confusions en valeurs absolues et en pourcentage

En observant les matrices, on se rend compte que la majorité des résultats prédits sont bons, à l'exception de certaines catégories où le modèle a eu plus de mal. Sur la matrice de confusion en pourcentages, les chiffres sur les diagonales sont tous supérieurs à 95% de sureté, ce qui indique que le modèle est bon.

Script de prédiction

Conformément aux exigences du cahier des charges, un script final en python a été développé. Il accepte en entrée les caractéristiques d'une borne sous forme de chaîne JSON ou via fichier JSON et renvoie le type d'implantation prédit ainsi que les probabilités associées à chaque classe. En cas de valeurs manquantes, elles sont complétées par des valeurs par défauts. Finalement, le script charge les quatre fichiers .pkl sauvegardés.

Besoin Client 4 : Modèle de prédiction du type d'implantation

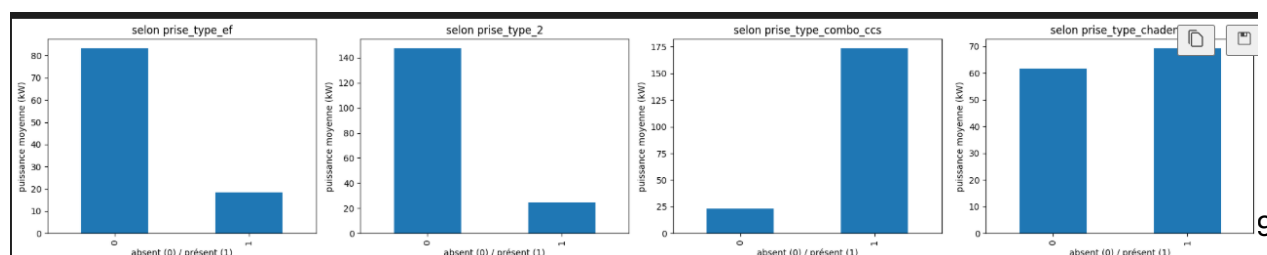
Préparation des données et choix des variables

Pour ce dernier besoin client 4, le but est de développer un modèle d'intelligence artificielle capable de prédire la puissance nominale d'une borne de recharge électrique à partir de ses caractéristiques. La puissance nominale étant une variable continue, elle a été transformée en variable catégorielle en quatre classes correspondant aux standards industriels de la recharge électrique : la charge lente (puissance inférieure ou égale à 7,4 kW), la charge accélérée (entre 7,4 et 22 kW), la charge rapide (entre 22 et 50 kW) et la charge ultra-rapide (supérieure à 50 kW).

Ce découpage permet de traiter le problème comme une classification multi-classes plutôt qu'une régression, ce qui est plus robuste face aux valeurs atypiques et plus interprétable pour un usage opérationnel. Un filtre a également été appliqué en amont pour écarter les valeurs aberrantes, en ne conservant que les bornes dont la puissance nominale est strictement positive et inférieure ou égale à 400 kW.

Les variables explicatives ont été sélectionnées en tenant compte de leur lien logique avec la puissance d'une borne. Les variables numériques retenues sont le nombre de points de charge (*nbre_pdc*), la longitude (*consolidated_longitude*) et la latitude (*consolidated_latitude*).

Les variables booléennes retenues sont les types de connecteurs disponibles (*prise_type_ef*, *prise_type_2*, *prise_type_combo_ccs*, *prise_type_chademo*, *prise_type_autre*), la gratuité de la recharge (*gratuit*) et la dédicace aux deux-roues (*station_deux_roues*). Ces variables sont particulièrement discriminantes : la présence d'une prise combo CCS ou CHAdeMO est quasi systématiquement associée à une charge rapide ou ultra-rapide, tandis qu'une prise de type EF correspond invariablement à une charge lente.

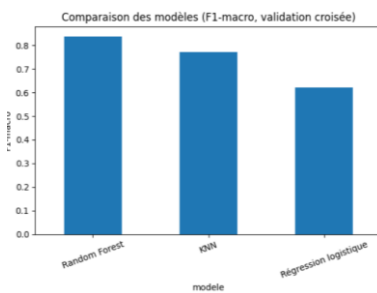


Présence des types de prise en fonction de la puissance nominale

Enfin, les variables catégorielles *implantation_station* et *condition_acces* ont été incluses car le contexte d'implantation est fortement lié à la puissance installée : une borne en parking public ou sur voirie sera plus souvent rapide qu'une borne en résidence privée.

Comme pour les autres besoins du client, les données ont été normalisées par conversion des chaînes de caractères en booléens et d'autres conversions. Tout ce prétraitement et le modèle ont été sauvegardés dans un fichier *modele_puissance.pkl* via la bibliothèque *joblib*.

Choix de l'algorithme



Trois algorithmes ont été évalués et comparés afin de retenir le modèle le plus performant. Les résultats obtenus en F1-macro par validation croisée sont les suivants : la Random Forest obtient un score de 0,835, le KNN (K plus proches voisins) obtient 0,768, et la régression logistique obtient 0,636.

F1 score par test de validation croisés

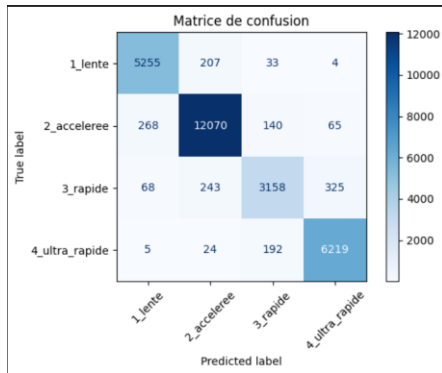
Le Random Forest a donc été retenu comme algorithme final. Comme expliqué avant, son principe repose sur la construction d'un grand nombre d'arbres de décision indépendants, chacun entraîné sur un sous-échantillon aléatoire des observations et utilisant à chaque nœud un sous-ensemble aléatoire des variables disponibles. La prédiction finale est obtenue par vote majoritaire de l'ensemble des arbres.

Les résultats inférieurs de la régression logistique s'expliquent par la nature non linéaire du lien entre les caractéristiques et la classe de puissance : par exemple, la combinaison d'une prise combo CCS avec une implantation sur voirie prédit bien mieux une charge ultra-rapide qu'une somme pondérée de ces variables ne pourrait le faire. Le KNN, bien que non linéaire, est sensible à la dimension élevée de l'espace de variables et à la présence de variables catégorielles encodée en binaire, ce qui limite ses performances. Le Random Forest gère nativement ces deux aspects, ce qui justifie sa supériorité sur ce jeu de données.

Métriques d'évaluation et analyse des résultats

La métrique principale retenue pour l'évaluation et la comparaison des modèles est le F1-macro, qui calcule la moyenne du F1-score de chaque classe sans tenir compte du déséquilibre éventuel entre les classes. Ce choix est justifié par le fait que les quatre classes de puissance ne sont pas nécessairement équilibrées dans le jeu de données : les bornes de charge accélérée sont par exemple bien plus nombreuses que les bornes ultra-rapides. Le F1-macro pénalise donc équitablement les erreurs sur les classes rares, contrairement à la simple exactitude qui serait biaisée par les classes majoritaires.

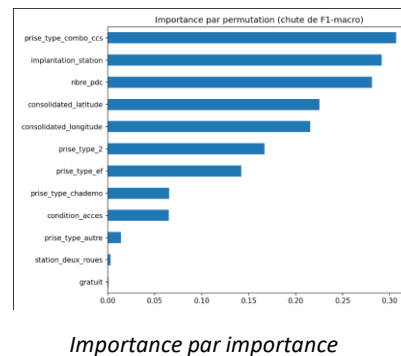
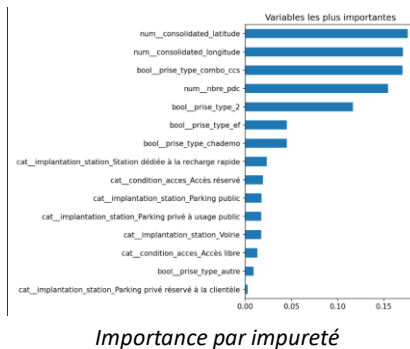
Les résultats obtenus sont présentés dans le tableau et les graphes ci-dessous.



En observant la matrice de confusion, on remarque que le modèle prédit plutôt bien à l'exception de la puissance rapide car c'est une tranche étroite et moins fréquente. Finalement, on obtient quand même une précision de 0.89 ce qui indique que le modèle se trompe rarement.

Importance des variables

Deux méthodes ont été utilisées pour analyser l'importance des variables. La première est l'importance par impureté, calculée directement à partir de la Random Forest : elle mesure la réduction moyenne de l'impureté de Gini apportée par chaque variable à travers tous les arbres. La seconde est l'importance par permutation : on mélange aléatoirement les valeurs d'une variable et on mesure la chute du F1-macro qui en résulte. Cette seconde méthode est plus fiable car l'importance par impureté tend à surestimer les variables continues comme la latitude et la longitude, et à diluer l'importance des variables catégorielles éclatées en plusieurs colonnes. Les graphiques correspondants sont présentés ci-dessous.



Script de prédiction

Un script de prédiction a été développé d'après les exigences du cahier des charges. Modele_puissance.pkl prend en entrée un dictionnaire contenant les caractéristiques d'une borne et retourne la classe prédite.

Conclusion

Ce projet nous a permis de mettre en pratique l'ensemble du pipeline d'un projet de Machine Learning appliqué à un jeu de données réel, en l'occurrence la base nationale des Infrastructures de Recharge pour Véhicules Électriques. En partant des données nettoyées et structurées lors de notre précédent projet de Big Data, nous avons abordé la visualisation géographique, l'apprentissage non supervisé et l'apprentissage supervisé, couvrant ainsi les trois grandes familles de méthodes attendues dans le cahier des charges.

Le besoin client 1 nous a permis de maîtriser la visualisation de données géolocalisées avec Folium, en produisant des cartes interactives et des cartes de chaleur permettant d'identifier visuellement la densité et la répartition des infrastructures sur le territoire métropolitain. Le besoin client 2 nous a confronté à la problématique du clustering non supervisé : le choix et la justification du nombre optimal de clusters via quatre métriques complémentaires nous ont appris à ne pas nous fier à un seul indicateur et à croiser les résultats pour prendre une décision éclairée. Les besoins clients 3 et 4 nous ont enfin initié à l'apprentissage supervisé, aussi bien en classification du type d'implantation qu'en classification de la puissance nominale. Dans les deux cas, la comparaison de plusieurs algorithmes et l'utilisation du GridSearchCV nous ont permis de sélectionner et d'optimiser un modèle de Random Forest, dont les performances se sont révélées satisfaisantes avec des précisions supérieures à 89%.

Sur le plan organisationnel, travailler en trinôme sur une période de cinq jours a nécessité une coordination. Le diagramme de Gantt, ainsi que Github et notre approche agile nous ont permis de paralléliser efficacement les tâches tout en maintenant une cohérence d'ensemble dans les choix techniques réalisés par chacun.

Des améliorations seraient envisageables pour prolonger ce travail. L'utilisation de DBSCAN pour le clustering permettrait de s'affranchir du choix préalable du nombre de clusters et de mieux gérer les zones de densité variable. Pour les modèles supervisés, l'intégration de davantage de variables issues de la base IRVE, ou l'exploration de modèles de Gradient Boosting, pourrait encore améliorer les performances prédictives. Finalement, les scripts des modèles seront utilisés pour la suite dans le prochain projet de Développement Web.